PROJECT: Mobile applications-Interaction design in vehicle

Smart Car Dashboard

Muhammad Ahmed Ali

1 Introduction

A glanceable, voice-aware dashboard built for the road

What It Is

- Android app connected to live vehicle data over MQTT
- Displays speed, RPM, and fuel level at a glance
- Car-friendly, high-contrast interface designed for quick glances, not reading

Target Users

- Driver — needs fast, glanceable info and a voice alert for anything urgent, without looking away from the road
- Any driver without a smart dashboard — benefits from retrofitting speed/RPM/fuel awareness and weather info onto an existing phone or tablet mount

2 Architecture

OBD Simulator
Python → publishes speed / RPM / fuel

→

MQTT Broker
Mosquitto → routes sensor messages

→

Android App
Subscribes → renders dashboard UI

→

Driver
Sees display → hears voice alerts

Text-to-Speech

Speaks the speed warning

Weather API

Live conditions for the driver

Git

Stores code and app history

3 Functions

Live Speed Gauge

Animated speedometer, color-coded green/yellow/red

RPM & Fuel Display

Compact live cards for engine RPM and fuel level

Voice Speed Warning

Spoken alert when the speed limit is exceeded

Visual Alert Banner

Red banner flashes briefly alongside the voice warning

Weather Card

Live temperature and conditions for the driver's area

Auto-Reconnect

MQTT client reconnects automatically if the link drops

4

Implementation

Tech Stack

- Kotlin, Android Studio
- Eclipse Paho MQTT client
- Mosquitto broker (local)
- Python 3 + paho-mqtt (simulator)
- Android TextToSpeech API
- OpenWeatherMap REST API

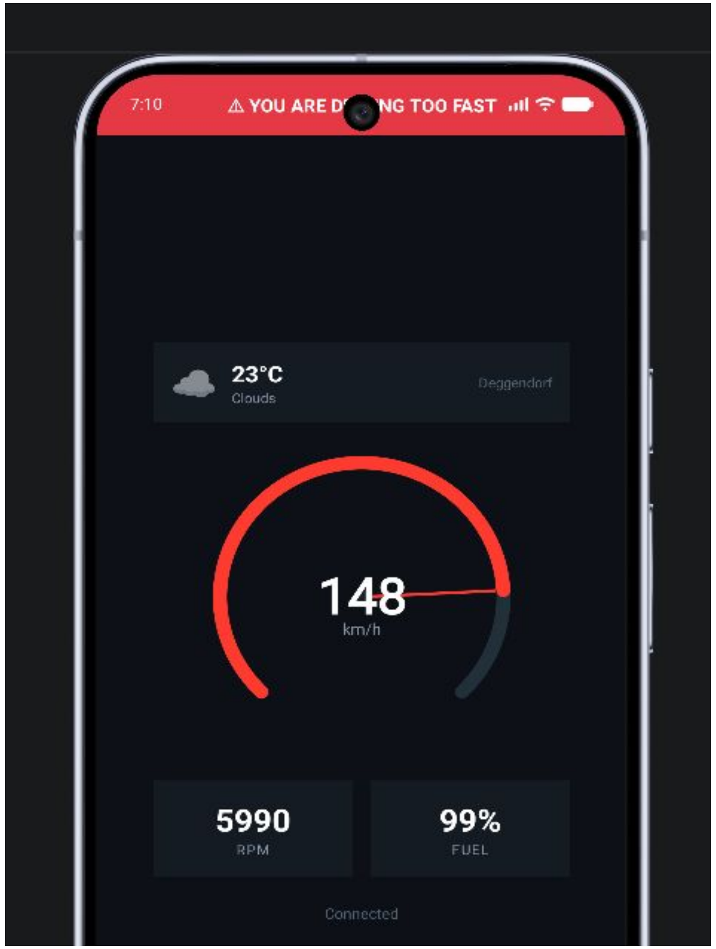
Key Components

- MainActivity: MQTT client, message handling, alert logic
- SpeedometerView: custom Canvas-drawn gauge with animation
- obd_simulator.py: publishes speed/RPM/fuel every second
- JSON payloads on car/speed, car/rpm, car/fuel topics
- Debounced logic prevents alert spam (8s cooldown)

Challenges Solved

- AppCompat theme mismatch on activity crash
- Mosquitto "local only mode" blocking real-device tests
- Emulator networking via 10.0.2.2 vs. real device IP
- Alert spam → solved with time-based debounce
- Async weather fetch off the main thread

4 Application



Conclusion

A functional automotive interaction prototype

- The project successfully demonstrates a car climate and weather monitoring interface running on the Android emulator.
- It includes working climate controls and live simulated sensor feedback.
- It addresses automotive interaction through readable controls, status colors and warnings.
- It can be evaluated through functional testing and a usability questionnaire.
- Future versions can connect the interface to CC3200 hardware and real vehicle sensor data.